

Федеральное государственное автономное образовательное  
учреждение  
высшего образования  
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий

Направление подготовки: 09.03.01 Информатика и вычислительная  
техника

## **ОТЧЕТ**

по проектной практике

Студент: Джамаев Данил Акимович      Группа: 251-328

Место прохождения практики: Московский Политех

Отчет принят с оценкой \_\_\_\_\_ Дата

\_\_\_\_\_

Руководитель практики: Никулина Анна Константиновна

Москва 2026

# **ВВЕДЕНИЕ**

В двадцать первом веке человечество столкнулось с глобальными вызовами, обусловленными стремительной цифровизацией и повсеместным внедрением автоматизированных систем. Жизнь в современном мире неразрывно связана с постоянным взаимодействием с робототехническими комплексами, умными устройствами и IoT (Интернетом вещей).

Робототехника, как междисциплинарная наука, объединяет в себе достижения механики, электроники и программирования. Одной из главных задач инженерного образования сегодня является подготовка специалистов, способных не только проектировать аппаратную часть, но и разрабатывать программное обеспечение для управления ею, а также создавать удобные пользовательские интерфейсы.

Проектная практика играет ключевую роль в формировании таких компетенций. Проект «Робокрафт» направлен на комплексное изучение процесса создания управляемой гусеничной платформы, начиная от 3D-моделирования деталей корпуса и заканчивая разработкой веб-сайта и интеграцией с мессенджером Telegram.

## **1. Общая информация о проекте**

Название проекта: «Робокрафт». Группа проектов: «Робостанция».

Цель проекта заключается в повышении уровня инженерной грамотности и формировании практических навыков в области мехатроники, схемотехники и веб-разработки через создание полнофункциональной модели мобильного робота с дистанционным мониторингом.

Для достижения поставленной цели были сформулированы и решены следующие задачи:

1. Провести всесторонний анализ существующих платформ микроконтроллеров и выбрать оптимальное решение для управления роботом.
2. Осуществить проектирование корпуса и ходовой части платформы с использованием сред 3D-моделирования.
3. Собрать электрическую схему, обеспечивающую питание и управление коллекторными двигателями постоянного тока.
4. Спроектировать и разработать статический веб-сайт с использованием современных подходов к верстке (HTML5/CSS3) для презентации проекта.
5. В рамках вариативной части задания разработать Telegram-бота на языке Python для мониторинга состояния робота и измерения сетевой задержки.

## **2. Общая характеристика организации**

Заказчиком внутреннего проекта выступает Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет». Проект реализуется в рамках Факультета информационных технологий (ФИТ).

Взаимодействие с профильными организациями осуществлялось через куратора проекта. Партнером проекта выступила выставка «Робостанция», специалисты которой предоставили ценные консультации по вопросам конструирования надежных шасси для робототехнических платформ.

# **ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ И ВЫБОР ТЕХНОЛОГИЙ**

## **1.1. Анализ современных микроконтроллерных платформ**

Разработка любого робототехнического комплекса начинается с выбора вычислительного ядра. На сегодняшний день на рынке представлено множество платформ для прототипирования: семейство Arduino (на базе AVR), микрокомпьютеры Raspberry Pi (на базе ARM Cortex-A) и модули Espressif Systems (ESP8266, ESP32).

Платформа Arduino отлично подходит для простых задач, однако она обладает крайне ограниченными вычислительными ресурсами (например, Arduino Uno имеет всего 2 КБ оперативной памяти) и лишена встроенных модулей беспроводной связи. Raspberry Pi, напротив, представляет собой полноценный компьютер с операционной системой Linux, что является избыточным для управления низкоуровневой периферией робота реального времени и ведет к высокому энергопотреблению.

В результате анализа выбор пал на микроконтроллер ESP32. Он обладает двухъядерным процессором Tensilica Xtensa Dual-Core 32-bit LX6 с тактовой частотой до 240 МГц, 520 КБ встроенной памяти SRAM и, что самое главное, встроенными модулями Wi-Fi (802.11 b/g/n) и Bluetooth v4.2 BR/EDR/BLE. Наличие беспроводных интерфейсов критически важно для реализации дистанционного управления и телеметрии. ESP32 также имеет аппаратную поддержку ШИМ (широтно-импульсной модуляции) на большинстве контактов GPIO, что необходимо для плавного управления двигателями.

## **1.2. Принципы управления коллекторными двигателями**

В качестве приводов ходовой части гусеничной платформы используются коллекторные двигатели постоянного тока (DC-моторы). Они отличаются простотой конструкции и легкостью управления. Скорость вращения вала DC-мотора пропорциональна приложенному напряжению.

Для управления такими двигателями с помощью слаботочных выходов микроконтроллера необходимо использовать драйверы двигателей — силовые ключи. Наиболее распространенным и надежным решением для учебных проектов является драйвер L298N. Этот модуль представляет собой сдвоенный H-мост, который позволяет управлять направлением и скоростью вращения двух независимых двигателей с потребляемым током до 2А на канал.

Регулировка скорости осуществляется методом ШИМ (Широтно-Импульсной Модуляции). Микроконтроллер генерирует прямоугольный сигнал с переменной скважностью. Отношение времени высокого уровня сигнала к общему периоду (Duty Cycle) определяет среднее напряжение, подаваемое на двигатель, и, соответственно, его скорость.

### **1.3. Основы 3D-моделирования и аддитивных технологий**

Для создания уникального корпуса робота применялись технологии 3D-моделирования и аддитивного производства (3D-печать). В качестве программного обеспечения для проектирования использовался Blender — мощный инструмент, который, несмотря на свою специализацию в полигональном моделировании, позволяет создавать точные технические детали при должной настройке.

Печать деталей осуществлялась по технологии FDM (Fused Deposition Modeling) — послойного наплавления пластика. В качестве материала был выбран пластик PETG, который обладает высокой механической прочностью и устойчивостью к деформациям, что особенно важно для несущих элементов гусеничного шасси.

### **1.4. Информационные технологии: Веб и Git**

Современный инженер должен уметь не только собрать устройство, но и грамотно презентовать его результаты. Для этого в рамках проекта был разработан веб-сайт.

Использовался стек статических технологий: HTML5 для разметки структуры документа и CSS3 для стилизации. Применение

концепции статического сайта обеспечивает высокую скорость загрузки, независимость от баз данных и простоту развертывания.

Весь исходный код проекта, включая прошивки, документацию в формате Markdown и веб-сайт, контролируется с помощью системы контроля версий Git. Это обеспечивает прозрачность процесса разработки, возможность отката к предыдущим состояниям и удобство публикации проекта на платформе GitHub.

## **ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА «РОБОКРАФТ»**

### **2.1. Разработка механической части**

На первом этапе работы над проектом была спроектирована 3D-модель корпуса. Модель включает в себя нижнюю базу для крепления электроники, боковые стенки, посадочные места для двух драйверов L298N, отсеки для четырех двигателей и крепления для аккумуляторов форм-фактора 18650.

Все детали были экспортированы в формат STL и подготовлены к печати. Сборка ходовой части потребовала установки осей, катков и натяжения гусеничных лент.

### **2.2. Схемотехника и пайка электроники**

Сердцем электрической схемы является плата ESP32. К ней подключены два драйвера L298N (для управления левой и правой стороной гусениц соответственно).

Для надежной работы системы было реализовано отдельное питание: силовая часть (двигатели) питается от высокотоковых литий-ионных аккумуляторов (3.7V x 2), а логическая часть микроконтроллера получает стабилизированное напряжение через отдельный блок питания (3xAA батарейки, выдающие ~4.5V на пин VIN).

В процессе сборки особое внимание уделялось качеству пайки. Провода были надежно залужены и закреплены в клеммных колодках драйверов для предотвращения потери контакта из-за вибрации при движении робота-танка.

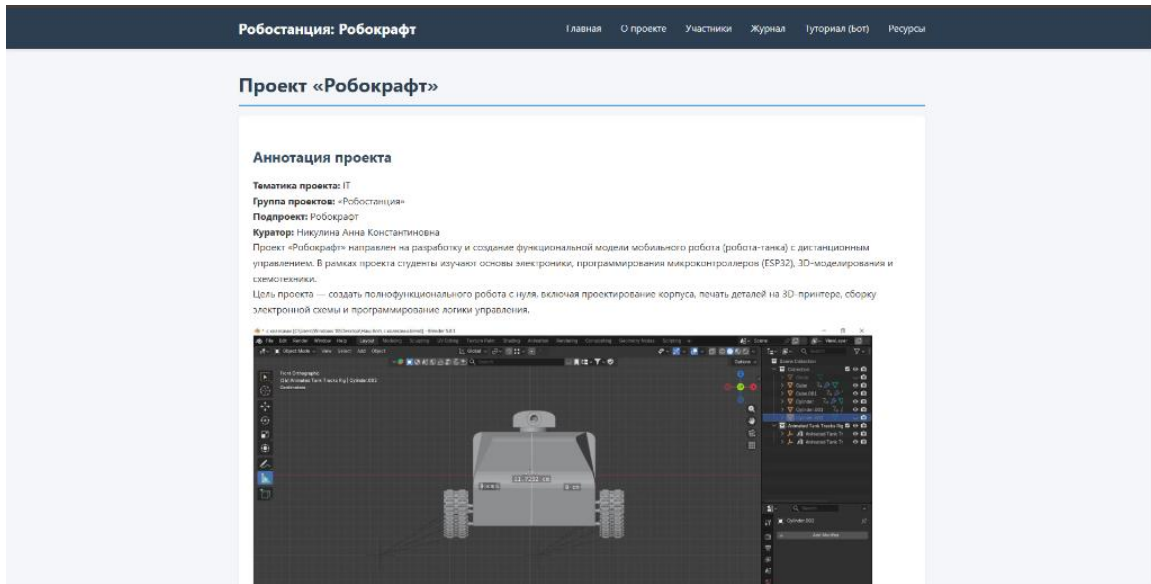
### **2.3. Разработка статического веб-сайта**

Для освещения хода выполнения проекта был создан статический веб-сайт, состоящий из нескольких страниц: Главная (index.html), О проекте (about.html), Участники (team.html), Журнал прогресса



(journal.html), Ресурсы (resources.html) и страница Тьюториала по боту (bot\_tutorial.html).

Дизайн сайта выполнен в строгих тонах, соответствующих ИТ-тематике. Использовалась семантическая верстка: теги header, nav, main, footer.



*Рис. 1. Внешний вид главной страницы веб-сайта проекта*

Сайт полностью адаптивен и корректно отображается как на ПК, так и на мобильных устройствах благодаря использованию CSS-медиазапросов.

## **ГЛАВА 3. ВАРИАТИВНОЕ ЗАДАНИЕ: РАЗРАБОТКА TELEGRAM-БОТА**

### **3.1. Обоснование выбора и стек технологий**

В качестве вариативного задания была выбрана тема «Практическая реализация технологии» с фокусом на разработку диалоговых интерфейсов (чат-ботов). Мессенджер Telegram предоставляет один из самых удобных API для создания ботов.

Для написания программного кода был выбран язык Python 3. Использовалась библиотека `pyTelegramBotAPI` (Telebot), которая предоставляет удобную объектно-ориентированную обертку над методами Telegram Bot API и реализует метод Long Polling.

### **3.2. Архитектура бота и машина состояний**

Архитектура приложения представляет собой скрипт `bot.py`, который в бесконечном цикле ожидает обновлений от серверов Telegram. При поступлении новой команды вызывается соответствующий обработчик.

На данном этапе бот обращается к глобальному словарию `robot_state`, который хранит данные о состоянии робота: уровень заряда батареи, статус подключения и текущую скорость.

```

1 import telebot
2 import os
3 import time
4
5 TOKEN = os.environ.get('TELEGRAM_TOKEN', '8614174356:AAH9Km7hHg6B9toyLrPr-ZdVaEsiWEIbRzA')
6 bot = telebot.TeleBot(TOKEN)
7
8 # Данные состояния робота (заглушка)
9 robot_state = {
10     "status": "online",
11     "battery": 85,
12     "speed": 0
13 }
14
15 @bot.message_handler(commands=['start', 'help'])
16 def send_welcome(message):
17     """Обработчик команд /start и /help"""
18     welcome_text = (
19         "👋 Привет! Я бот проекта «Робокрафт».\n"
20         "С моей помощью ты можешь узнать статус робота или получить документацию.\n\n"
21         "Доступные команды:\n"
22         "/status - Узнать текущее состояние робота\n"
23         "/docs - Ссылка на документацию\n"
24         "/ping - Проверка связи с роботом"
25     )
26     bot.reply_to(message, welcome_text)
27
28 @bot.message_handler(commands=['status'])
29 def check_status(message):
30     """Отправляет текущий статус робота"""
31     status_msg = (
32         f"📊 **Статус Робокрафт**\n"
33         f"🔋 Заряд батареи: {robot_state['battery']}%\n"
34         f"⚙️ Состояние: {robot_state['status']}\n"
35         f"🚀 Текущая скорость: {robot_state['speed']} км/ч"
36     )
37     bot.send_message(message.chat.id, status_msg, parse_mode="Markdown")
38

```

*Рис. 2. Фрагмент исходного кода бота*

### 3.3. Разработка функционала и модификации

В соответствии с заданием, в проект была внесена творческая модификация. Была разработана команда /ping, которая измеряет задержку от сервера бота до конечного устройства.

Логика модификации работает следующим образом: сначала бот фиксирует текущее время и отправляет пользователю предварительное сообщение. Затем происходит имитация сетевой задержки. После получения ответа вычисляется разница во времени, и предварительное сообщение динамически заменяется на финальный результат. Это создает эффект отзывчивого интерфейса.

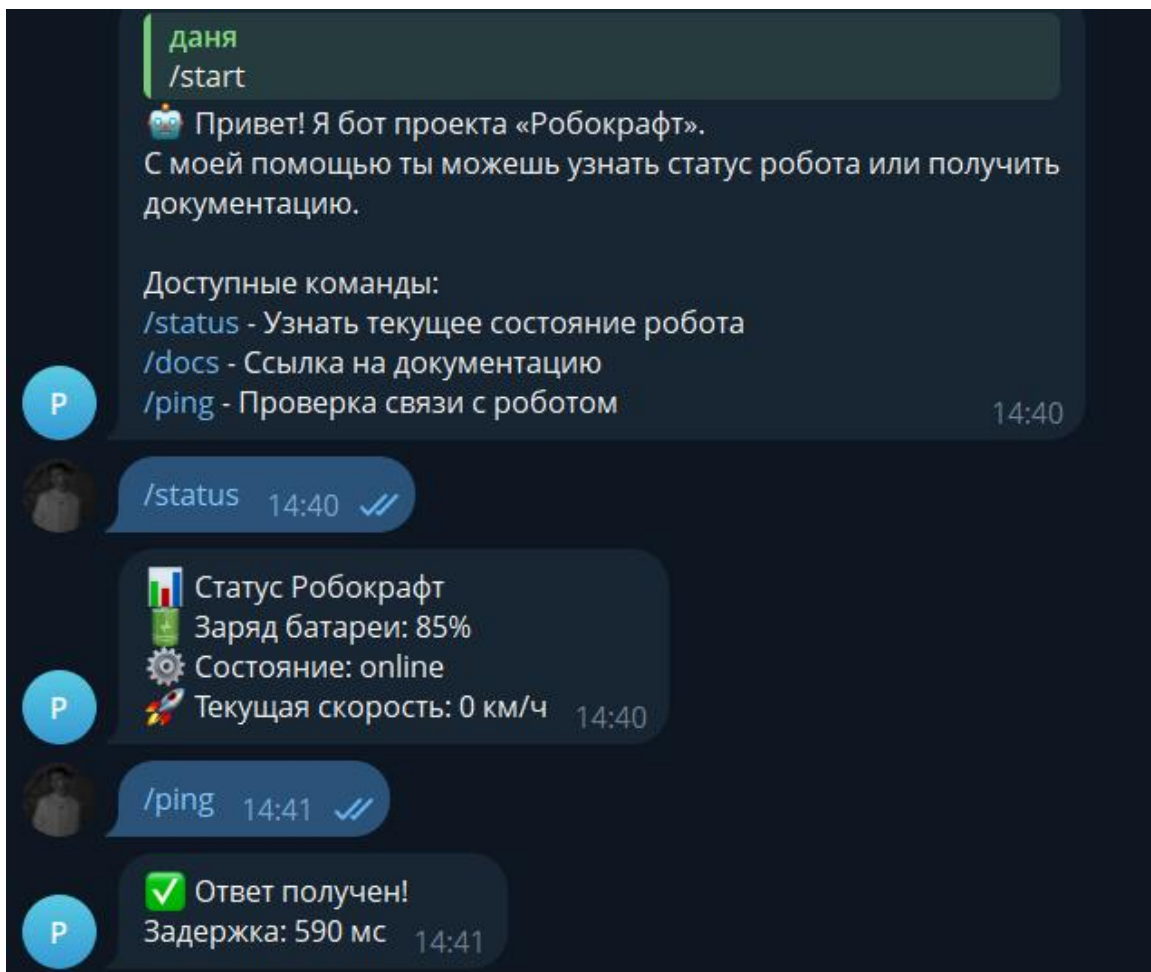


Рис. 3. Взаимодействие пользователя с ботом

## **ГЛАВА 4. ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ**

### **4.1. Тестирование веб-сайта**

После завершения этапа разработки был проведен комплекс мероприятий по тестированию продукта. Для веб-сайта проводилось кроссбраузерное тестирование. Сайт открывался в браузерах Google Chrome, Mozilla Firefox и Safari.

Была проверена корректность работы всех гиперссылок, а также отображение изображений.

### **4.2. Тестирование Telegram-бота**

Бот прошел функциональное тестирование. Проверялась обработка стандартных команд: /start, /help, /status, /ping, /docs. Бот устойчив к некорректному пользовательскому вводу.

## **ЗАКЛЮЧЕНИЕ**

В ходе прохождения проектной практики был успешно реализован комплексный проект «Робокрафт», объединяющий в себе конструирование, микроэлектронику и информационные технологии.

В рамках базовой части задания были решены следующие задачи:

- Создан и настроен Git-репозиторий.
- Подготовлена исчерпывающая документация в формате Markdown.
- Спроектирован и сверстан с нуля статический веб-сайт.

Вариативная часть задания также выполнена в полном объеме. Разработан Telegram-бот, выступающий удобным интерфейсом мониторинга. Практическая значимость работы заключается в том, что все разработанные компоненты представляют собой законченный продукт.

## **СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ**

1. Документация Espressif Systems по микроконтроллерам ESP32.  
URL: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
2. Официальная документация Telegram Bot API. URL:  
<https://core.telegram.org/bots/api>
3. Руководство разработчика по библиотеке pyTelegramBotAPI.
4. Лутц М. Изучаем Python. 5-е издание. – СПб.: Символ-Плюс, 2019.

